

Azure Services Required for Manual Deployment

1 Comment / By Dot Net Tutorials / April 13, 2026

Back to: [Microsoft Azure Tutorials](#)

Azure Services Required for Manual Deployment

In the previous chapters, we learned what deployment means, how to prepare our ASP.NET Core Web API project for Azure deployment, and how Azure basics, such as accounts, subscriptions, and resource groups, work. Now, before we actually deploy our application, we need to understand which Azure services are required.

This is a very important step because many beginners open the Azure portal and see many services, but do not know which service is used for what. For manual deployment of an ASP.NET Core Web API with SQL Server, we need only a few important Azure services. Once you understand the purpose of each service, the deployment process becomes much easier.

Why Do We Need Azure Services for Deployment?

When we run our ASP.NET Core Web API on the local machine, it works only on that computer. We can test it locally using Swagger or Postman, but other users cannot access it over the internet.

To make the application available online, we need cloud services. Azure provides these services. One service hosts the API, another stores the database, another provides the server resources, and another helps us organize everything properly.

So, before deploying, we must first understand which Azure services are needed and what role each plays.

What Is Manual Deployment?

Manual deployment means we create and configure the Azure resources ourselves, step by step. At this stage, we are not using advanced deployment options such as:

- Docker
- Kubernetes
- CI/CD pipelines
- GitHub Actions
- Azure DevOps pipelines

Instead, we follow the traditional deployment approach to clearly understand what each Azure service does.

In manual deployment, we usually do these steps ourselves:

- Create the Resource Group

- Create the App Service Plan
- Create the App Service
- Create the Azure SQL Logical Server
- Create the Azure SQL Database
- Configure the connection string
- Publish the application manually

This is actually the best way for beginners to learn Azure deployment because it builds a strong foundation. Once the manual process is clear, advanced deployment methods become much easier to understand.

Main Azure Services Required for Manual Deployment

For manual deployment of our ASP.NET Core Web API with SQL Server, the main Azure services we need are:

- Resource Group
- Azure App Service
- App Service Plan
- Azure SQL Database
- Azure SQL Logical Server

Let us now understand each one in a simple way.

1. Resource Group

A Resource Group is a logical container in Azure that groups related resources. In simple words, it is like a folder in Azure. Just as we keep related files inside one folder on our computer, Azure allows us to keep related cloud resources inside one Resource Group.

For example, if our project is ProductManagementApi, then one Resource Group may contain:

- App Service
- App Service Plan
- Azure SQL Database
- Azure SQL Logical Server

The Resource Group itself does not run the application or store database data. Its job is only to organize related Azure resources in one place. So, in simple terms: **Resource Group = a container that groups related Azure resources**. This makes management easier because all project resources stay in one place.

2. Azure App Service

Azure App Service is the Azure service that hosts and runs our ASP.NET Core Web API in the cloud. Right now, when we run the API locally, it works only on our machine. But when we publish it to Azure App Service, the API becomes available online through a public URL.

That means:

- users can access it,
- websites can call it,
- mobile apps can use it,
- and other systems can connect to it.

So, in simple words: **Azure App Service = the place where our Web API runs online**

What does Azure App Service do?

It helps us:

- Host the ASP.NET Core Web API.
- Get a public URL.
- Deploy from Visual Studio or a ZIP package.
- Configure application settings.
- Manage logs and diagnostics.

If someone asks, “Where is your API actually running in Azure?” the answer is: **Azure App Service**.

3. App Service Plan

Many beginners confuse between Azure App Service and App Service Plan, so this is an important concept to understand.

- The **App Service** is where the application runs.
- The **App Service Plan** provides the server resources needed to run that application.

It decides things such as:

- CPU
- Memory
- Pricing tier
- Region
- Scaling options

So, if Azure App Service is where the application is hosted, then the App Service Plan is the underlying infrastructure.

You can think of it like this:

- **App Service** = The shop
- **App Service Plan** = The building, electricity, and space needed to run the shop

Without the building and power, the shop cannot function. Similarly, without the App Service Plan, the App Service cannot run. So, in simple words: **App Service Plan = the compute capacity and pricing setup behind the App Service**

4. Azure SQL Database

Our Web API usually needs data. That data may relate to products, users, categories, orders, employees, or other business information. To store that data in Azure, we use **Azure SQL Database**.

Azure SQL Database is the cloud-based database service provided by Microsoft Azure. It is similar to SQL Server, but it is managed by Azure in the cloud.

It stores:

- Tables
- Rows and columns
- Relationships
- Views
- Stored procedures
- And other database objects

So, in simple words: **Azure SQL Database = the cloud database where our application data is stored**. This is the database that our ASP.NET Core Web API will connect to for reading and writing data.

5. Azure SQL Logical Server

Before creating an Azure SQL Database, Azure asks us to create something called a **Logical Server**. A Logical Server is the parent container for one or more Azure SQL Databases. It helps manage:

- Server name
- Admin username and password
- Firewall rules
- Access settings

This means:

- The **Logical Server** manages the database environment.
- The **SQL Database** stores the actual tables and data.

You can think of it like this:

- **Azure SQL Logical Server** = Apartment building
- **Azure SQL Database** = One flat inside that building

Without the building, the flat cannot exist.

So, in simple words: **Azure SQL Logical Server = the parent database server that manages access and security for Azure SQL Databases.** One Azure SQL Logical Server can contain multiple Azure SQL Databases.

Important Note:

As a beginner, you do not need to learn every Azure service before deployment. For this manual deployment chapter, focus only on the services directly required to host the API and database.

So, at this stage, you can safely ignore services like:

- Azure Kubernetes Service
- Azure Functions
- Azure Container Registry
- Load Balancers
- Service Bus
- Azure DevOps

Those are useful later, but they are not required for this basic manual deployment.

Summary

In this chapter, we understood the main Azure services required for manual deployment of an ASP.NET Core Web API with SQL Server.

We learned that:

- **Resource Group** → Keeps related Azure resources together.
- **App Service Plan** → Provides server power and pricing setup.
- **App Service** → Runs the Web API.
- **Azure SQL Logical Server** → Manages database server settings and access.
- **Azure SQL Database** → Stores the application data.

So, before starting the hands-on deployment, we must first be comfortable with these services. Once these concepts are clear, the actual deployment process becomes much easier and more beginner-friendly.



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET

Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information